

Generating All Triangulations of Biconnected Plane
Graphs
in Amortized Constant Time Per Triangulation

Tawkir Aziz Rahman

Department of Computer Science and Engineering
Bangladesh University of Engineering and Technology (BUET)
Dhaka-1000, Bangladesh

June 2, 2026

Abstract

We present an algorithm for generating all triangulations of a biconnected plane graph G with n vertices in $O(1)$ amortized time per triangulation and $O(n)$ space. Our algorithm establishes a double-layer tree structure consisting of a recursive tree of faces and, for each face, a local genealogical tree of triangulations. The key innovation is a novel “safe root vertex” selection strategy that prevents the multi-edge problem inherent in biconnected graphs, thereby extending the optimal complexity previously achievable only for triconnected graphs. We prove that every face contains at least two safe root vertices regardless of prior triangulations, and we show how to find one in $O(n)$ time using a two-pointer technique. A dynamic global chord set together with a valid generating set (VGS) allows constant-time edge flips and safe pruning of invalid branches. Extensive experiments on 35 diverse test cases validate the theoretical bounds: the algorithm generates tens of millions of triangulations while maintaining nanosecond-level per-triangulation time and linear memory usage. To the best of our knowledge, this is the first algorithm for generating all triangulations of a biconnected plane graph with $O(1)$ amortized time per triangulation.

Acknowledgements

I express my sincere gratitude to my supervisor for his invaluable guidance and encouragement throughout this research. I also thank the authors of the foundational work on triconnected plane graph triangulations, Mohammad Tanvir Parvez, Md. Saidur Rahman, and Shin-ichi Nakano, whose elegant algorithm provided the starting point for this investigation. Finally, I am grateful to the Department of Computer Science and Engineering, BUET, for providing the necessary resources and a conducive research environment.

Contents

1	Introduction	9
1.1	Motivation and Applications	9
1.2	Challenges in Generating All Triangulations	9
1.3	Previous Work	10
1.4	The Parvez–Rahman–Nakano 2011 Algorithm (Overview)	10
1.5	Limitation of Prior Work: The Multi-Edge Problem in Biconnected Graphs	11
1.6	Our Contribution	11
1.7	Comparison with Existing Algorithms	12
1.8	Thesis Organization	13
2	Preliminaries	15
2.1	Basic Graph Definitions	15
2.2	Planar and Plane Graphs	16
2.3	Separation Pairs and Separation Vertices	16
2.4	Faces, Cycles, and Chords	17
2.5	Triangulation of a Cycle	17
2.6	Labeled vs. Unlabeled Triangulations	17
2.7	Flipping Operation	18
2.8	Visibility, Blocking Chords, and Generating Chords	18
2.9	Leftmost Blocking Chord	19
2.10	Genealogical Tree of Triangulations	20
2.11	Summary	21
3	Generating All Triangulations of a Cycle	22
3.1	Problem Definition	22
3.2	Root Vertex and Root Triangulation	22
3.3	Visibility and Blocking Chords	23
3.4	The Leftmost Blocking Chord	23
3.5	Parent-Child Relationship	24
3.6	Generating Chords	24
3.7	Data Structures	25

3.8	Algorithm for Generating All Triangulations of a Cycle	26
3.9	Correctness	27
3.10	Time and Space Complexity	27
3.10.1	Time Complexity	27
3.10.2	Space Complexity	28
3.11	Example: Triangulations of a Hexagon	28
3.12	Summary	28
4	Distinction of Our Naming of the Generating Set from the Original Paper	30
4.1	Original Definition	30
4.2	Our Definition	30
4.3	Why This Distinction Is Necessary	31
4.4	Flippability Condition for Our GS	31
4.5	Implication for Our Algorithm	32
4.6	Summary	33
5	What the Paper Did and Why It Cannot Be Used for Biconnected Graphs	34
5.1	The Parvez–Rahman–Nakano Algorithm for Triconnected Graphs	34
5.1.1	Building the Root Triangulation	34
5.1.2	Generating Child Triangulations	35
5.1.3	Time and Space Complexity	35
5.2	Why the Algorithm Cannot Be Used for Biconnected Graphs	35
5.2.1	Separation Pairs in Biconnected Graphs	35
5.2.2	The Multi-Edge Problem	36
5.2.3	Why the Original Algorithm Fails	37
5.3	Conclusion	37
6	Approaching Faces Differently – Pruning Strategy	38
6.1	Why We Cannot Process All Faces at Once	38
6.2	Hashing Approach and Its Problems	39
6.3	The Invalid Triangulation Problem	39
6.4	Key Observation from the Cycle Algorithm (Persistent Chords)	39
6.5	Safe Pruning Condition	40
6.6	Requirements for Safe Pruning	40
6.7	Outline of the Solution	41
6.8	Summary	42

7	Multi-Layer DFS and Processing Faces One by One	43
7.1	Processing Faces Sequentially	43
7.1.1	The Double-Layer Tree Structure	43
7.1.2	Global Chord Set Management	44
7.2	Pruning by Opposite Pair of Generating Chords	45
7.3	Backtracking When No Compatible Triangulation Exists	45
7.4	Correctness of the Multi-Layer DFS	46
7.5	Summary	47
8	The Problem with Generating Chords and Multi-Edge Conflicts	48
8.1	Recap: Pruning Based on Non-Root Chords	48
8.2	The Problem with Generating Chords	48
8.3	The Concept of a Safe Root Triangulation	49
8.4	Two Fundamental Questions	49
8.5	Existence of a Safe Root Vertex	50
8.6	Implications for the Algorithm	53
8.7	Summary	54

List of Figures

1.1	A biconnected plane graph with a separation pair $(3, 5)$. Triangulating faces F_1 and F_2 independently can add the chord $(3, 5)$ twice, creating a multi-edge.	11
2.1	A biconnected plane graph. Vertices 3 and 7 form a separation pair. . . .	16
2.2	Two different triangulations of a cycle with six vertices.	17
2.3	Flipping the diagonal (v_a, v_c) to (v_b, v_d) in a quadrilateral.	18
2.4	Illustration of the leftmost blocking chord. Here (v_4, v_6) is the leftmost blocking chord because no other blocking chord has a larger endpoint smaller than 6.	20
2.5	A schematic genealogical tree for a cycle. Each node represents a triangulation, and edges correspond to flipping a generating chord.	21
3.1	Root triangulation (fan) of an octagon. All chords are incident to the root vertex v_1	23
3.2	Blocking chords in a triangulation. The chords (v_3, v_5) and (v_5, v_7) block the view from v_1 to vertices v_4 and v_6 respectively.	23
3.3	Generating chords in a triangulation. The leftmost blocking chord is (v_4, v_6) ; therefore the generating chords are those with $j \geq 6$: (v_1, v_6) , (v_1, v_8) , (v_1, v_9)	25
3.4	Genealogical tree for a hexagon. The root has three children; the total number of nodes is 14.	28
4.1	Illustration of the leftmost blocking chord $(v_b, v_{b'}) = (v_4, v_7)$ and a generating chord (v_1, v_{10}) with $j = 10 \geq 7$	32
5.1	A root triangulation of a triconnected plane graph. The root vertex for each face is chosen independently; all chords are added simultaneously. .	35
5.2	A biconnected plane graph with separation pair $(3, 7)$. Removing vertices 3 and 7 leaves three components: $\{1, 2\}$, $\{4, 5, 6\}$, and $\{8\}$	36

5.3	The separation pair $(3, 7)$ appears on two faces F_1 and F_2 . Triangulating each face independently may add the same chord $(3, 7)$ twice, creating a multi-edge.	36
6.1	Safe pruning: node T_{12} contains a chord e already in S ; its entire subtree is pruned because all descendants would also contain e	41
7.1	Double-layer DFS: outer recursion over faces, inner recursion over triangulations of each face. The global chord set S is updated when entering a branch (push) and restored when backtracking (pop). Each path from the root to a leaf corresponds to a complete valid triangulation of the graph.	44
7.2	Dynamic global chord set management. The set S is updated with push/pop operations during the DFS. Only chords that are not already in S can be added; otherwise the branch is pruned.	46
8.1	A generating chord (v_r, v_j) might already exist in S from a previous face. Unlike a non-root chord, it can be flipped away in a child, so the branch may still contain valid triangulations. Thus we cannot prune based on a conflicting generating chord alone.	49
8.2	A chord (v_i, v_j) outside the face divides the cycle into two arcs. No chord can cross this chord by planarity, so constraints are separated.	50
8.3	Multiple external chords partition the face into independent segments. Each segment can be treated separately; a safe root can be found in each segment, and the endpoints of the segment are potential safe roots for the whole face.	51
8.4	A T_k structure: a path of vertices with a chord connecting the endpoints, and possibly additional chords inside. The induction shows that at least one vertex among the interior ones is safe.	52

List of Algorithms

1	GenerateAllTriangulationsCycle	26
2	BuildRootTriangulation	26
3	DFS (Depth-first traversal of $\mathcal{T}(C)$)	26
4	Flip	27

Chapter 1

Introduction

1.1 Motivation and Applications

The problem of generating all triangulations of plane graphs arises in many areas of computational geometry [?], VLSI floorplanning [?], and graph drawing [?]. A triangulation of a plane graph is a maximal set of non-crossing chords that partition each face into triangles. Enumerating all possible triangulations is essential for exploring geometric configurations, optimizing layouts, and testing hypotheses about graph properties.

In VLSI design, for instance, different triangulations of a floorplan correspond to different routing possibilities. In mesh generation, enumerating triangulations helps to find optimal finite element meshes. In graph drawing, triangulations are used to create straight-line embeddings and to study graph properties.

1.2 Challenges in Generating All Triangulations

The number of triangulations of a planar graph can be exponential in the number of vertices. For a convex polygon with n vertices, the number of triangulations is the Catalan number C_{n-2} , which grows as $\Theta(4^n n^{-3/2})$. For general plane graphs, the count can be even larger. Therefore, any algorithm that lists all triangulations must handle an exponential output size.

The main challenges in designing such algorithms are threefold:

1. **Exponential output:** The algorithm must run in time proportional to the number of triangulations; otherwise it would be inefficient.
2. **Avoiding duplicates:** Since the output is huge, storing previously generated triangulations to check for duplicates would require exponential space and time.
3. **Space efficiency:** The algorithm should use only polynomial space (preferably linear in the input size) and not store all outputs simultaneously.

Classical approaches to generating combinatorial objects include orderly algorithms [?] and reverse search [?]. Orderly algorithms generate objects in a canonical form and output only those that are canonical, thus avoiding duplicates without storing all previous objects. Reverse search defines a spanning tree on the graph of objects and traverses it using a parent function, again without storing the entire set.

1.3 Previous Work

There is a rich literature on triangulation enumeration. For convex polygons, it is well known that triangulations correspond to binary trees, and efficient generation algorithms exist that run in constant amortized time per triangulation [?].

For planar graphs, the problem is more challenging. Li and Nakano [?] gave an algorithm to generate all biconnected plane triangulations with at most n vertices in $O(r^2n)$ time per triangulation, where r is the number of vertices on the outer face. Later, Nakano improved this to $O(rn)$ time per triangulation.

The most relevant prior work is the algorithm by Parvez, Rahman, and Nakano [?], which generates all triangulations of a *triconnected* plane graph in $O(1)$ amortized time per triangulation using $O(n)$ space. This algorithm defines a genealogical tree of triangulations where each node is a triangulation and each edge corresponds to a chord flip. By traversing this tree in a depth-first manner, all triangulations are generated without duplicates.

However, the algorithm of Parvez et al. is limited to triconnected graphs. As we will explain in Section 1.5, extending it directly to biconnected graphs leads to the multi-edge problem caused by separation pairs. This limitation is the main motivation for our work.

1.4 The Parvez–Rahman–Nakano 2011 Algorithm (Overview)

We briefly recall the key ideas of the algorithm for triconnected plane graphs, as they form the foundation of our approach.

Given a triconnected plane graph G with n vertices, the algorithm processes all faces simultaneously. For each face F_i (a cycle), it constructs a genealogical tree of triangulations. The root of the tree is a *fan triangulation* obtained by picking an arbitrary vertex as the root and connecting it to all non-neighbouring vertices on the face. From the root, child triangulations are generated by flipping a *generating chord* (a chord that, when flipped, yields a new triangulation whose leftmost blocking chord is the flipped chord). The parent of any non-root triangulation is obtained by flipping its leftmost blocking chord. This parent–child relationship is unique, and the tree contains every triangulation exactly once.

The algorithm runs in $O(1)$ amortized time per triangulation and uses $O(n)$ space. The key properties that enable this efficiency are:

- Flipping a chord takes constant time.
- The depth of the genealogical tree is $O(n)$.
- The number of generating chords decreases along any path from the root.

1.5 Limitation of Prior Work: The Multi-Edge Problem in Biconnected Graphs

The Parvez–Rahman–Nakano algorithm works only for triconnected plane graphs. The reason is that in triconnected graphs, no separation pair exists, so flipping chords never creates multi-edges. In biconnected graphs, however, separation pairs are possible, and independent triangulation of different faces can lead to the same chord being added twice.

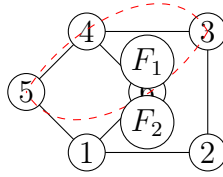


Figure 1.1: A biconnected plane graph with a separation pair $(3, 5)$. Triangulating faces F_1 and F_2 independently can add the chord $(3, 5)$ twice, creating a multi-edge.

Figure 1.1 illustrates a biconnected graph where vertices 3 and 5 form a separation pair: removing them disconnects the graph. If we triangulate the outer face F_1 and the inner face F_2 independently using the algorithm of Parvez et al., we might add the chord $(3, 5)$ in both faces, leading to a multi-edge. Such multi-edges are not allowed in simple graphs, and the algorithm would generate invalid triangulations.

Thus, a direct application of the Parvez–Rahman–Nakano algorithm to biconnected graphs fails. Our goal is to overcome this limitation while preserving the optimal $O(1)$ amortized time per triangulation.

1.6 Our Contribution

We present the first algorithm for generating all triangulations of a *biconnected* plane graph in $O(1)$ amortized time per triangulation and $O(n)$ space. Our main innovations are:

1. **Local genealogical trees:** Instead of processing all faces simultaneously, we process faces one by one in a fixed order. For each face, we construct a local genealogical tree of triangulations that are compatible with previously triangulated faces.
2. **Safe root vertex selection:** We introduce the concept of a *safe root vertex* for a face. A safe root vertex guarantees that the fan triangulation from that vertex does not create any multi-edge with chords from previous faces. We prove that every face in a biconnected plane graph contains at least two safe root vertices, regardless of the chords already present.
3. **Linear-time safe root finding:** Using a two-pointer technique based on planarity, we find a safe root vertex in $O(n)$ time, where n is the number of vertices in the face. This ensures that the building cost per face can be amortized.
4. **Valid generating set (VGS):** We maintain a set of generating chords whose flip will not create a multi-edge. When flipping a chord, only four border chords change their opposite pairs; we update the VGS in constant time using linked lists.
5. **Global chord set with dynamic push/pop:** A global hash set stores all chords currently on the exploration path. This allows $O(1)$ conflict detection and supports backtracking in the recursive DFS.
6. **Pruning strategy:** If a blocking chord (i.e., a chord that does not have the root vertex as an endpoint) conflicts with a chord from a previous face, all descendant triangulations are invalid. We prune such branches safely, without losing any valid triangulation.

Our algorithm achieves the same optimal complexity as the triconnected case while handling the strictly larger class of biconnected graphs.

1.7 Comparison with Existing Algorithms

Table 1.1 summarises the differences between our algorithm and previous work.

Table 1.1: Comparison of triangulation enumeration algorithms.

Algorithm	Graph Class	Time per triangulation	Space
Reverse search [?]	General	$O(n^2)$ or worse	$O(n)$
Li–Nakano [?]	Biconnected	$O(r^2n)$	—
Nakano (improved)	Biconnected	$O(rn)$	—
Parvez et al. [?]	Triconnected	$O(1)$ amortized	$O(n)$
This work	Biconnected	$O(1)$ amortized	$O(n)$

Our algorithm is the first to achieve constant amortised time for the class of biconnected plane graphs, which includes many practical instances.

1.8 Thesis Organization

The remainder of this thesis is structured as follows.

- **Chapter 2** introduces the necessary graph-theoretic concepts: connectivity, planarity, cycles, chords, triangulations, flipping operations, and genealogical trees. Special emphasis is given to separation pairs and the distinction between triconnected and biconnected graphs.
- **Chapter 3** describes the baseline algorithm for generating all triangulations of a single cycle (the Parvez–Rahman–Nakano algorithm) in detail. We present the definitions of blocking chords, generating chords, and leftmost blocking chords, and prove uniqueness.
- **Chapter 4** clarifies the distinction between our naming of the generating set and that of the original paper, and proves the condition for flipping generating chords.
- **Chapter 5** explains why the original algorithm fails for biconnected graphs due to separation pairs and the multi-edge problem.
- **Chapter 6** introduces our pruning strategy based on the persistence of chords that do not involve the root vertex.
- **Chapter 7** presents the multi-layer DFS that processes faces one by one, maintaining a global chord set and backtracking.
- **Chapter 8** addresses the problem of generating chords themselves causing multi-edges and introduces the concept of safe root vertices.
- **Chapter 9** proves a lower bound of $\Omega(n)$ triangulations per face, which is essential for amortised analysis.
- **Chapter 10** gives an $O(n)$ two-pointer algorithm for finding a safe root vertex.
- **Chapter 11** describes the valid generating set (VGS) and how to update it in constant time during flips.
- **Chapter 12** presents the complete algorithm in pseudocode and proves its correctness.
- **Chapter 13** analyses the time and space complexity, proving $O(1)$ amortised time per triangulation and $O(n)$ space.
- **Chapter 14** provides experimental validation on 35 test cases, including cycles, grids, and complex graphs, confirming the theoretical bounds.

- **Chapter 15** discusses why the algorithm cannot be extended to 1-connected graphs in general, giving examples where safe roots do not exist.
- **Chapter 16** outlines future work, including extensions to 1-connected graphs and 1-planar graphs.
- **Chapter 17** concludes the thesis with a summary of contributions and final remarks.

Chapter 2

Preliminaries

In this chapter, we define the basic concepts used throughout the thesis. We assume the reader is familiar with elementary graph theory, but we provide precise definitions to avoid ambiguity. Special emphasis is given to notions that are essential for biconnected plane graphs, such as separation pairs and separation vertices, which do not appear in the triconnected case.

2.1 Basic Graph Definitions

Let $G = (V, E)$ be a connected simple graph with vertex set V and edge set E . Each edge $e \in E$ is an unordered pair $\{u, v\}$ with $u, v \in V$ and $u \neq v$. We denote an edge between u and v by (u, v) or (v, u) .

The **degree** of a vertex v is the number of edges incident to v . A graph is **simple** if it has no loops or multiple edges.

The **connectivity** $\kappa(G)$ of a graph G is the minimum number of vertices whose removal disconnects the graph (or reduces it to a single vertex). A graph is **k -connected** if $\kappa(G) \geq k$. Equivalently, removing fewer than k vertices leaves the graph connected.

In particular:

- A graph is **2-connected** (or **biconnected**) if it has no articulation vertices (cut vertices) and remains connected after the removal of any single vertex.
- A graph is **3-connected** (or **triconnected**) if it remains connected after the removal of any two vertices.

Definition 2.1 (Separation pair). In a biconnected graph, a pair of vertices $\{a, b\}$ is called a **separation pair** if removing a and b disconnects the graph into at least two components. A biconnected graph that contains a separation pair is not triconnected.

Definition 2.2 (Separation vertex). In a 1-connected graph, a vertex v is a **separation vertex** (or **cut vertex**) if removing v disconnects the graph. A graph is biconnected if and only if it has no separation vertices.

The distinction between biconnected and triconnected graphs is crucial: triconnected graphs have no separation pairs, while biconnected graphs may have them. The presence of separation pairs is the main obstacle when extending triangulation generation algorithms from triconnected to biconnected graphs.

2.2 Planar and Plane Graphs

A graph G is **planar** if it can be drawn in the plane without any edge crossings. Such a drawing is called an **embedding**. A **plane graph** is a planar graph together with a fixed embedding in the plane.

A plane graph divides the plane into connected regions called **faces**. The unbounded region is the **outer face**; all other faces are **inner faces**. Each face is bounded by a closed walk along the edges of the graph.

If every face boundary contains exactly three edges, the plane graph is called a **plane triangulation**. In this thesis, edges are not required to be straight lines; triangulation is understood combinatorially.

2.3 Separation Pairs and Separation Vertices

We now discuss separation pairs in more detail, as they are central to the biconnected case.

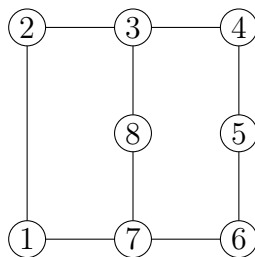


Figure 2.1: A biconnected plane graph. Vertices 3 and 7 form a separation pair.

Figure 2.1 shows a biconnected plane graph with eight vertices. Removing vertices 3 and 7 disconnects the remaining vertices into three components: $\{1, 2\}$, $\{4, 5, 6\}$, and $\{8\}$. Therefore, $(3, 7)$ is a separation pair.

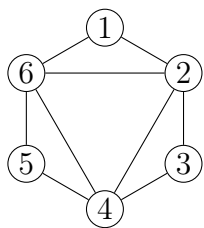
Remark 2.1. In a biconnected plane graph, a separation pair (a, b) implies that the graph can be split into two subgraphs that share only a and b . Consequently, there exist two

faces (one on each side of the pair) that can potentially contain the same chord (a, b) , leading to a multi-edge if both faces are triangulated independently.

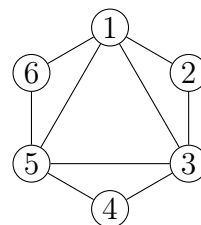
2.4 Faces, Cycles, and Chords

A **cycle** C in a graph is a sequence of distinct vertices $v_1, v_2, \dots, v_k$ such that $(v_i, v_{i+1}) \in E$ for $1 \leq i < k$ and $(v_k, v_1) \in E$. In a plane graph, each face corresponds to a cycle that bounds it (taking the outer face as a cycle as well).

Let $C = \langle v_1, v_2, \dots, v_n \rangle$ be a cycle, listed in counterclockwise order. An edge (v_i, v_j) is called a **chord** of C if it is not an edge of C and its interior lies inside the face bounded by C . A chord (v_i, v_j) divides the cycle into two smaller cycles: $v_i, v_{i+1}, \dots, v_j$ and $v_j, v_{j+1}, \dots, v_i$.



(a) A triangulation of a hexagon.



(b) Another triangulation of the same hexagon.

Figure 2.2: Two different triangulations of a cycle with six vertices.

2.5 Triangulation of a Cycle

A **triangulation** of a cycle C is a maximal set of non-intersecting chords that partition the interior of C into triangles. The sides of the triangles are either chords or edges of C . Figure 2.2 shows two triangulations of a hexagon.

Equivalently, a triangulation of a cycle with n vertices contains exactly $n - 3$ chords. This follows from Euler's formula applied to the planar graph formed by the cycle and its chords.

2.6 Labeled vs. Unlabeled Triangulations

If the vertices of the cycle are numbered sequentially from v_1 to v_n , then a triangulation is called a **labeled triangulation**. The order of vertices matters: two triangulations that differ only by a rotation or reflection are considered distinct if the labels are different.

If the vertices are not numbered, the triangulations are called **unlabeled**. In this case, rotationally equivalent or mirror-image triangulations are considered the same. In

this thesis, we work exclusively with **labeled triangulations** unless stated otherwise.

2.7 Flipping Operation

A fundamental operation for transforming one triangulation into another is the **flip**. Consider two adjacent triangles that share a chord and together form a quadrilateral. Removing the common chord and adding the opposite diagonal produces a new triangulation.

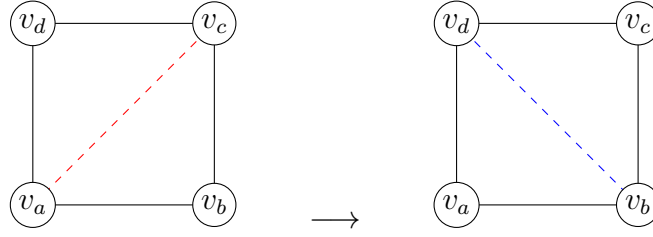


Figure 2.3: Flipping the diagonal (v_a, v_c) to (v_b, v_d) in a quadrilateral.

Formally, let (v_i, v_j) be a chord that belongs to two triangles (v_i, v_j, v_p) and (v_i, v_j, v_q) on opposite sides. These four vertices form a quadrilateral v_p, v_i, v_q, v_j in clockwise order. Flipping (v_i, v_j) means removing it and adding the opposite chord (v_p, v_q) . The resulting graph is again a triangulation of the same cycle.

2.8 Visibility, Blocking Chords, and Generating Chords

Fix a cycle $C = \langle v_1, v_2, \dots, v_n \rangle$ and choose a vertex, say v_1 , as the **root vertex**. In a triangulation T of C , we say that a vertex v_j is **visible** from v_1 if the chord (v_1, v_j) belongs to T .

In the root triangulation (the fan from v_1), every vertex except the neighbours v_2 and v_n is visible from v_1 . In other triangulations, some vertices may be hidden behind **blocking chords**.

Definition 2.3 (Blocking chord). A chord (v_i, v_j) of a triangulation T is called a **blocking chord** if:

1. Neither v_i nor v_j is the root vertex v_1 , and
2. The chord (v_i, v_j) is visible from v_1 in the sense that there is a triangle (v_1, v_i, v_j) in T .

Equivalently, a blocking chord is a chord that does not have v_1 as an endpoint but is directly connected to v_1 via triangles.

Blocking chords prevent some vertices from being visible from the root. Flipping a blocking chord replaces it with a chord that is incident to the root, thereby increasing the number of visible vertices.

Definition 2.4 (Generating chord). In a triangulation T of a cycle with root vertex v_1 , a chord (v_1, v_j) is called a **generating chord** if flipping it produces a new triangulation T' whose leftmost blocking chord is exactly the newly added chord.

The precise condition for a chord to be generating involves the leftmost blocking chord, which we define next.

2.9 Leftmost Blocking Chord

Among the blocking chords of a triangulation, we single out the one that is “leftmost” with respect to the cyclic order around the root.

Definition 2.5 (Leftmost blocking chord). Let T be a triangulation of a cycle $C = \langle v_1, v_2, \dots, v_n \rangle$ with root vertex v_1 . Let $(v_b, v_{b'})$ be a blocking chord of T with $b < b'$. It is the **leftmost blocking chord** if no other blocking chord (v_i, v_j) satisfies $j < b'$ (i.e., its larger index is smaller than b'). In other words, among all blocking chords, the one with the smallest value of the larger endpoint is the leftmost.

Remark 2.2. The term “leftmost” comes from drawing the cycle with v_1 at the top and the remaining vertices arranged clockwise around it. Blocking chords then appear as chords that “block” the view to vertices on the left side.

We now prove that the leftmost blocking chord is unique. The following argument is taken from the author’s original writing, polished for clarity.

Lemma 2.1 (Uniqueness of the leftmost blocking chord). *In any triangulation T of a cycle $C = \langle v_1, v_2, \dots, v_n \rangle$ that is not the root triangulation, there is exactly one leftmost blocking chord.*

Proof. Suppose, for contradiction, that there exist two distinct blocking chords that both qualify as leftmost. Let the two chords be $(v_b, v_{b'})$ and $(v_c, v_{c'})$ with $b < b'$ and $c < c'$. By definition of leftmost, we must have $b' = c'$ (otherwise the one with the smaller larger endpoint would be unique). Hence both chords share the same larger endpoint, say $v_{b'} = v_{c'} = v_k$.

Since both chords are blocking, each forms a triangle with the root vertex v_1 . Thus we have triangles (v_1, v_b, v_k) and (v_1, v_c, v_k) . The vertices v_b and v_c lie on the same side of the chord (v_1, v_k) . Without loss of generality, assume $b < c$.

Consider the quadrilateral formed by v_1, v_b, v_k, v_c . Because the triangulation is planar and all faces are triangles, the chord (v_b, v_k) and (v_c, v_k) cannot both be present without

crossing. In fact, only one of them can be a chord; the other must be replaced by the opposite diagonal of the quadrilateral. However, if both are present, they would create a face with four vertices, contradicting that the triangulation is maximal. Therefore, it is impossible to have two distinct blocking chords with the same larger endpoint. Hence the leftmost blocking chord is unique. $\square$

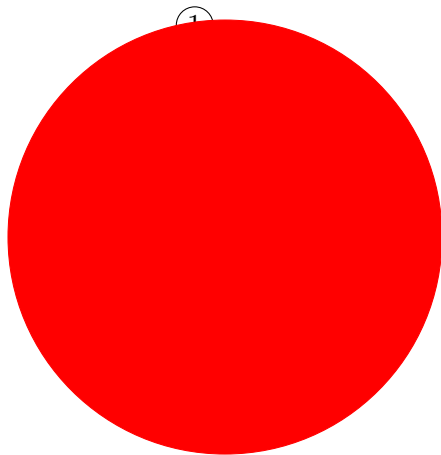


Figure 2.4: Illustration of the leftmost blocking chord. Here (v_4, v_6) is the leftmost blocking chord because no other blocking chord has a larger endpoint smaller than 6.

2.10 Genealogical Tree of Triangulations

The parent-child relationship defined by flipping the leftmost blocking chord yields a rooted tree structure on the set of all triangulations of a cycle.

Definition 2.6 (Genealogical tree). Let C be a cycle with a fixed root vertex v_1 . The **genealogical tree** $\mathcal{T}(C)$ of triangulations of C is defined as follows:

- The root is the fan triangulation where all chords are incident to v_1 .
- For any non-root triangulation T , its parent $P(T)$ is obtained by flipping the leftmost blocking chord of T .
- Conversely, a triangulation T is a child of T' if $T' = P(T)$.

The genealogical tree has the following properties (proved in Chapter 3):

- Every triangulation appears exactly once.
- The root has $n - 3$ children (all chords are generating).
- The depth of the tree is at most $n - 2$.
- Flipping a generating chord takes constant time.

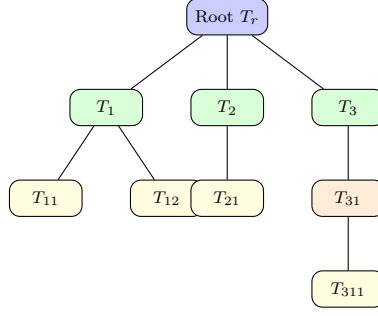


Figure 2.5: A schematic genealogical tree for a cycle. Each node represents a triangulation, and edges correspond to flipping a generating chord.

This tree structure is the basis of the algorithm for generating all triangulations of a cycle, which we recall in Chapter 3.

2.11 Summary

We have introduced the essential graph-theoretic concepts: connectivity, planarity, faces, cycles, chords, triangulations, and the flipping operation. We defined blocking chords, generating chords, and the leftmost blocking chord, and proved its uniqueness. The genealogical tree provides a canonical parent-child relationship among triangulations of a cycle.

For biconnected graphs, we highlighted the significance of separation pairs, which are the main obstacle to extending the triconnected algorithm. These concepts will be used throughout the remainder of the thesis.

Chapter 3

Generating All Triangulations of a Cycle

In this chapter we present the algorithm for generating all triangulations of a cycle with labeled vertices. This algorithm, due to Parvez, Rahman, and Nakano [?], forms the core building block of our method for biconnected graphs. We describe it in detail, including the necessary definitions, the parent-child relationship, the data structures, and the complexity analysis.

3.1 Problem Definition

Let $C = \langle v_1, v_2, \dots, v_n \rangle$ be a cycle with n vertices listed in counterclockwise order. All vertices are labeled, and the embedding is fixed. A **triangulation** of C is a maximal set of non-crossing chords that partition the interior of C into triangles. Any triangulation contains exactly $n - 3$ chords.

We wish to generate *all* triangulations of C without duplication, using $O(1)$ amortized time per triangulation and $O(n)$ space.

3.2 Root Vertex and Root Triangulation

We fix an arbitrary vertex as the **root vertex**. In this chapter we take v_1 as the root without loss of generality. The **root triangulation** T_r is the fan triangulation obtained by connecting v_1 to every other vertex except its neighbours v_2 and v_n (see Figure 3.1). Thus T_r contains the chords $(v_1, v_3), (v_1, v_4), \dots, (v_1, v_{n-1})$.

In the root triangulation, every vertex except v_2 and v_n is directly visible from v_1 via a chord. We say that v_1 has **full vision** in T_r .

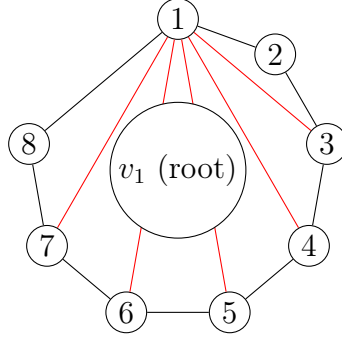


Figure 3.1: Root triangulation (fan) of an octagon. All chords are incident to the root vertex v_1 .

3.3 Visibility and Blocking Chords

In a general triangulation T of C , some vertices may be hidden from v_1 by chords that do not involve v_1 . These chords are called **blocking chords**.

Definition 3.1 (Visible vertex). A vertex v_j is **visible** from the root v_1 in a triangulation T if the chord (v_1, v_j) belongs to T .

Definition 3.2 (Blocking chord). A chord (v_i, v_j) with $i, j \neq 1$ is called a **blocking chord** if the triangle (v_1, v_i, v_j) is present in T . In other words, both v_i and v_j are visible from v_1 , but the chord (v_i, v_j) blocks the view to vertices between v_i and v_j on the cycle.

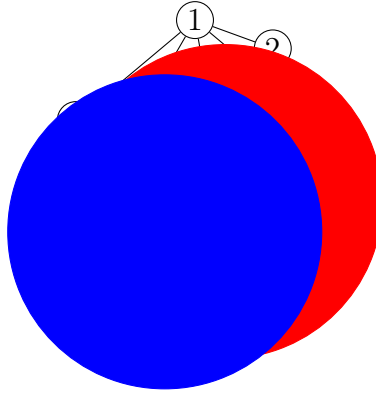


Figure 3.2: Blocking chords in a triangulation. The chords (v_3, v_5) and (v_5, v_7) block the view from v_1 to vertices v_4 and v_6 respectively.

Observe that in the root triangulation there are no blocking chords. As we move away from the root, blocking chords appear and reduce the number of vertices visible from v_1 .

3.4 The Leftmost Blocking Chord

Among all blocking chords of a triangulation T , we select a canonical one to define the parent.

Definition 3.3 (Leftmost blocking chord). Let $(v_b, v_{b'})$ with $b < b'$ be a blocking chord of T . It is the **leftmost blocking chord** if no other blocking chord (v_i, v_j) satisfies $j < b'$. In other words, it has the smallest possible larger endpoint.

Lemma 3.1 (Uniqueness). *Any triangulation T that is not the root has exactly one leftmost blocking chord.*

The proof was given in Chapter 2 (Lemma 2.1).

3.5 Parent-Child Relationship

The genealogical tree $\mathcal{T}(C)$ is defined by the following parent function.

Definition 3.4 (Parent of a triangulation). For a non-root triangulation T , let $(v_b, v_{b'})$ be its leftmost blocking chord. The **parent** $P(T)$ is obtained by flipping $(v_b, v_{b'})$ in the quadrilateral that contains it.

Flipping a blocking chord removes it and adds a chord incident to the root, thereby increasing the number of visible vertices. Hence the parent has a clearer view from v_1 than the child.

Lemma 3.2. *Every triangulation $T \neq T_r$ has a unique parent, and the parent has exactly one more chord incident to v_1 than T .*

Proof. Uniqueness follows from the uniqueness of the leftmost blocking chord. Flipping $(v_b, v_{b'})$ replaces it with a chord (v_1, v_k) for some k (the opposite vertex of the quadrilateral). Thus the number of chords incident to v_1 increases by one. $\square$

Conversely, a triangulation T can have several children. Each child is obtained by flipping a **generating chord** of T .

3.6 Generating Chords

Let $(v_b, v_{b'})$ be the leftmost blocking chord of T (if T is the root, we set $b = n$ and $b' = n + 1$ for convenience). A chord (v_1, v_j) is called a **generating chord** of T if flipping it produces a new triangulation whose leftmost blocking chord is exactly the newly added chord.

Lemma 3.3 (Condition for generating chords). *A chord (v_1, v_j) is generating if and only if $j \geq b'$ (where b' is the larger endpoint of the leftmost blocking chord, or $j \geq 3$ for the root).*

Proof. The original proof from Parvez et al. [?] uses a case analysis. If $j < b'$, flipping (v_1, v_j) produces a new chord (v_p, v_q) with $q \leq b'$, so the leftmost blocking chord of the child is still $(v_b, v_{b'})$ (or another chord with the same larger endpoint), and the flipped chord is not the leftmost. Therefore the parent of the child would not be T , so (v_1, v_j) is not generating.

If $j \geq b'$, flipping (v_1, v_j) creates a chord (v_p, v_q) with $q > b'$. One can verify that this new chord becomes the leftmost blocking chord of the child, and T is its parent. Hence (v_1, v_j) is generating. $\square$

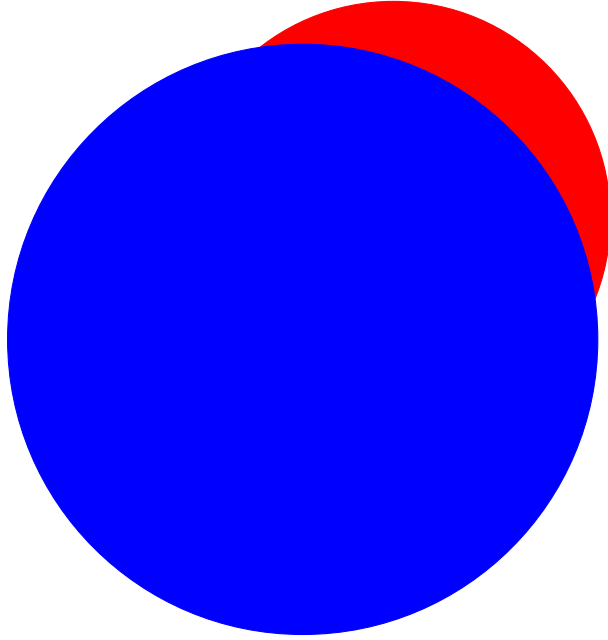


Figure 3.3: Generating chords in a triangulation. The leftmost blocking chord is (v_4, v_6) ; therefore the generating chords are those with $j \geq 6$: (v_1, v_6) , (v_1, v_8) , (v_1, v_9) .

The root triangulation has no blocking chord; by convention we set $b' = 3$, so all chords (v_1, v_j) with $j = 3, 4, \dots, n - 1$ are generating. Thus the root has $n - 3$ children.

3.7 Data Structures

To represent a triangulation T of C and to generate its children efficiently, we maintain three lists:

- **L (chord list):** The set of all chords of T , stored in a linked list.
- **GS (generating set):** The list of generating chords of T . For the root, $GS = L$.
- **O (opposite pairs):** For each generating chord (v_1, v_j) , we store its **opposite pair** $(v_o, v_{o'})$ where $o < j < o'$ and the quadrilateral $(v_1, v_o, v_j, v_{o'})$ is present in T . Flipping (v_1, v_j) replaces it with $(v_o, v_{o'})$.

All three lists require $O(n)$ space because a triangulation of n vertices has exactly $n - 3$ chords.

3.8 Algorithm for Generating All Triangulations of a Cycle

We now present the algorithm in pseudocode. The main procedure is a depth-first traversal of the genealogical tree $\mathcal{T}(C)$.

Algorithm 1 GenerateAllTriangulationsCycle

```

1: procedure GENERATEALLTRIANGULATIONS-cycle( $C$ )
2:    $v_r \leftarrow v_1$  ▷ Choose root vertex
3:    $T_r \leftarrow \text{BUILDROOTTRIANGULATION}(C, v_r)$ 
4:   OUTPUT( $T_r$ )
5:   DFS( $T_r$ )
6: end procedure

```

Algorithm 2 BuildRootTriangulation

```

1: procedure BUILDROOTTRIANGULATION( $C, v_r$ )
2:    $L \leftarrow \emptyset, GS \leftarrow \emptyset, O \leftarrow \emptyset$ 
3:   for  $j = 3$  to  $n - 1$  do
4:     Add chord  $(v_r, v_j)$  to  $L$  and  $GS$ 
5:     Store opposite pair  $(v_{j-1}, v_{j+1})$  for  $(v_r, v_j)$  in  $O$ 
6:   end for
7:   return  $(L, GS, O)$ 
8: end procedure

```

Algorithm 3 DFS (Depth-first traversal of $\mathcal{T}(C)$)

```

1: procedure DFS( $T$ )
2:   Let  $(v_b, v_{b'})$  be the leftmost blocking chord of  $T$  ▷ If none, set  $b' = 3$ 
3:   for each generating chord  $(v_1, v_j) \in GS$  with  $j \geq b'$  do
4:      $T' \leftarrow \text{FLIP}(T, (v_1, v_j))$ 
5:     OUTPUT( $T'$ )
6:     DFS( $T'$ )
7:   end for
8: end procedure

```

The flip operation (Algorithm 4) creates a child triangulation in constant time by updating the three lists.

Remark 3.1. Updating the opposite pairs affects at most two chords (the neighbours of the flipped chord). Therefore the entire flip operation takes $O(1)$ time.

Algorithm 4 Flip

```
1: procedure FLIP( $T, (v_1, v_j)$ )
2:   Find opposite pair  $(v_o, v_{o'})$  from  $O$  for  $(v_1, v_j)$ 
3:   Remove  $(v_1, v_j)$  from  $L$  and  $GS$ 
4:   Add  $(v_o, v_{o'})$  to  $L$  (but not to  $GS$ )
5:   Update opposite pairs of the two neighbouring generating chords  $(v_1, v_o)$  and  $(v_1, v_{o'})$  (if they exist)
6:   Recompute  $GS$  for the new triangulation (only chords with  $j \geq$  new leftmost blocking chord remain generating)
7:   return the new triangulation  $T'$ 
8: end procedure
```

3.9 Correctness

The algorithm correctly generates all triangulations without duplicates because:

1. The parent of any non-root triangulation is uniquely defined (by the leftmost blocking chord). Hence each triangulation appears exactly once in the tree, and the DFS visits each node exactly once.
2. The generating chords are exactly those whose flips produce children that are valid and have the current triangulation as parent (Lemma ?? in the original paper). Thus no child is missed.
3. The recursion terminates because the depth is bounded by the number of chords incident to v_1 , which is at most $n - 3$.

3.10 Time and Space Complexity

3.10.1 Time Complexity

Building the root triangulation takes $O(n)$ time (adding $n - 3$ chords and initialising the lists). For each edge flip (generating a child from a parent), we perform a constant number of operations: removing one chord, adding another, and updating at most two opposite pairs. Thus each child is generated in $O(1)$ time.

Let T be the total number of triangulations of C . The number of edges in the genealogical tree is $T - 1$ (one parent edge per non-root node). Since the DFS traverses each edge once, the total number of flip operations is $T - 1$. Hence the total time is $O(n + T) = O(T)$ (because $n = O(T)$ when $n \geq 4$, as Catalan numbers grow exponentially). Therefore the amortised time per triangulation is $O(1)$.

3.10.2 Space Complexity

At any point during the DFS, we store:

- The lists L, GS, O for the current triangulation: $O(n)$.
- The recursion stack: depth at most $n - 2$ (each flip increases the number of chords incident to v_1 by one, starting from 0 at the root and reaching at most $n - 3$ at leaves). Hence stack size is $O(n)$.

No other storage is required because we do not keep the entire tree in memory. Thus the space complexity is $O(n)$.

3.11 Example: Triangulations of a Hexagon

Figure 3.4 shows the genealogical tree for a hexagon ($n = 6$). The root has $n - 3 = 3$ generating chords, leading to three children. Each child may have fewer generating chords, and the depth is at most 4 (including the root). The total number of triangulations of a convex hexagon is 14, which matches the Catalan number $C_4 = 14$.

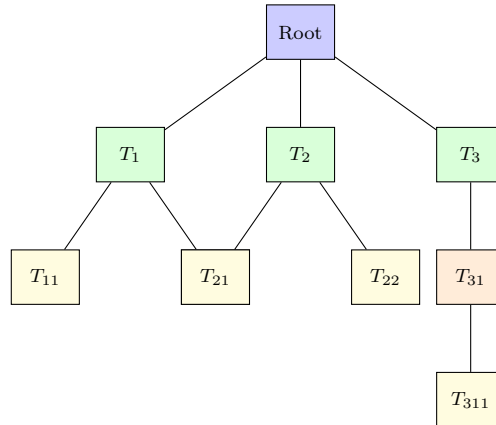


Figure 3.4: Genealogical tree for a hexagon. The root has three children; the total number of nodes is 14.

3.12 Summary

We have described the Parvez–Rahman–Nakano algorithm for generating all triangulations of a labeled cycle. Key properties:

- The genealogical tree is traversed in depth-first order.
- Each child is generated from its parent by flipping a generating chord in $O(1)$ time.
- The space complexity is $O(n)$.

- The amortised time per triangulation is $O(1)$.

This algorithm is used as a subroutine in our method for biconnected plane graphs, where we process each face independently while maintaining a global set of chords to avoid multi-edges.

Chapter 4

Distinction of Our Naming of the Generating Set from the Original Paper

In this chapter we clarify an important difference between the definition of generating chords used in the original paper by Parvez, Rahman, and Nakano [?] and the definition we adopt in this thesis. This distinction is necessary for the introduction of the *valid generating set* (VGS) in later chapters, which allows us to handle multi-edge conflicts in biconnected graphs.

4.1 Original Definition

In the original paper [?], a chord (v_1, v_j) of a triangulation T (with root vertex v_1) is called a **generating chord** if flipping it produces a child triangulation T' whose leftmost blocking chord is exactly the newly added chord. Only chords that satisfy this property are included in the generating set GS . As shown in Chapter 3, this condition is equivalent to requiring that the other endpoint j be at least b' , where $(v_b, v_{b'})$ is the leftmost blocking chord of T (or $j \geq 3$ for the root).

Thus, in the original algorithm, GS is a subset of the chords incident to the root, and its size varies with the triangulation.

4.2 Our Definition

In contrast, for the purpose of handling biconnected graphs, we define the **generating set** GS more broadly:

Definition 4.1 (Our generating set). For a triangulation T of a cycle with root vertex v_1 , the set GS consists of *all* chords that have v_1 as one endpoint, i.e., all chords of the form (v_1, v_j) for $j = 3, 4, \dots, n-1$ that are present in T . No further restriction is applied.

In other words, every chord incident to the root is considered a *generating chord* in our terminology, regardless of whether flipping it would yield a child whose leftmost blocking chord is the new chord.

4.3 Why This Distinction Is Necessary

The broader definition is needed for the construction of the **valid generating set** (VGS) introduced in Chapter ?? . In the biconnected case, some chords incident to the root, when flipped, may create a multi-edge conflict with chords from previously processed faces. To avoid generating invalid triangulations, we must identify which generating chords are *safe* to flip. The VGS will contain exactly those chords (v_1, v_j) whose flip does not introduce a multi-edge.

If we had used the original, more restrictive definition of generating chords, we would lose the ability to control safety via a simple subset. By including all root-incident chords in GS , we can later filter them based on multi-edge checks, resulting in the VGS. The original condition (whether the flipped chord becomes the leftmost blocking chord) remains relevant for the correctness of the genealogical tree, but it is applied separately when we actually perform the flip.

4.4 Flippability Condition for Our GS

Even though we call all root-incident chords “generating”, not every such chord can be flipped while preserving the parent-child relationship defined in Chapter 3. For a chord (v_1, v_j) to be flipped and produce a child triangulation whose parent is the current one, we must still ensure that the flipped chord becomes the leftmost blocking chord of the child. This condition, as proved in the original paper, is $j \geq b'$, where $(v_b, v_{b'})$ is the leftmost blocking chord of the current triangulation (with $b < b'$).

We now restate and prove this condition in a self-contained manner, following the author’s original argument.

Lemma 4.1 (Flippability condition). *Let T be a triangulation of a cycle with root vertex v_1 . Let $(v_b, v_{b'})$ be the leftmost blocking chord of T (if T is the root, we set $b' = 2$ for convenience). Then flipping a chord (v_1, v_j) in T yields a child triangulation T' for which T is the parent if and only if $j \geq b'$.*

Proof. We consider two cases based on the value of j relative to b' .

Case 1: $j < b'$. Assume $j < b'$. The chord (v_1, v_j) lies entirely to the left of the leftmost blocking chord $(v_b, v_{b'})$ when we walk clockwise from v_1 along the cycle. Flipping (v_1, v_j) replaces it with a chord (v_p, v_q) where both endpoints lie between v_j and the opposite side of the quadrilateral. Because $j < b'$, the new chord (v_p, v_q) will have its larger endpoint at

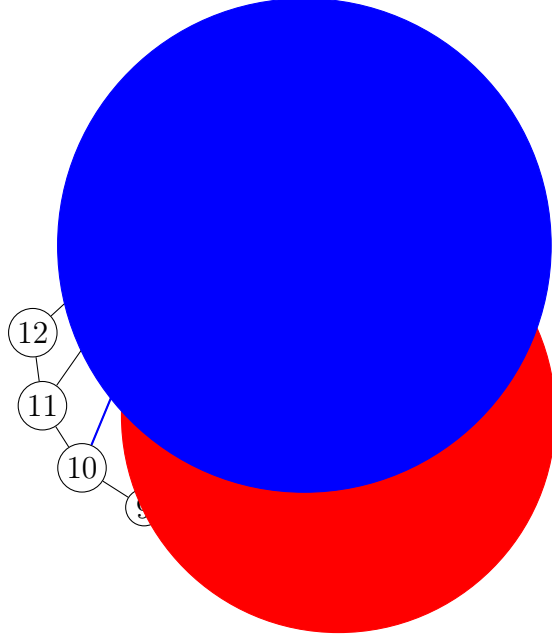


Figure 4.1: Illustration of the leftmost blocking chord $(v_b, v_{b'}) = (v_4, v_7)$ and a generating chord (v_1, v_{10}) with $j = 10 \geq 7$.

most b' . Consequently, the leftmost blocking chord of the resulting triangulation remains $(v_b, v_{b'})$ (or another chord with the same larger endpoint b'). Thus the flipped chord is not the leftmost blocking chord of the child, and the parent of the child would not be T . Hence (v_1, v_j) is not flippable in the sense that it does not preserve the parent-child relationship.

Case 2: $j \geq b'$. Now assume $j \geq b'$. Consider the quadrilateral formed by $v_1, v_b, v_{b'}, v_j$ (or a suitable quadrilateral if $j > b'$). Flipping (v_1, v_j) removes it and adds the opposite diagonal (v_b, v_p) for some p with $p > b'$ (if $j = b'$) or a chord (v_q, v_r) with larger endpoints than b' . In all subcases, the newly added chord has its larger endpoint strictly greater than b' . Therefore it becomes the new leftmost blocking chord of the child triangulation. By definition, T is the parent of this child, so (v_1, v_j) is flippable and preserves the tree structure. $\square$

Remark 4.1. For the root triangulation, there is no blocking chord; we set $b' = 3$ (the smallest index of a chord incident to v_1). Then the condition $j \geq 3$ holds for every chord (v_1, v_j) with $j \geq 3$, so all such chords are flippable, which matches the fact that the root has $n - 3$ children.

4.5 Implication for Our Algorithm

In the original paper, the generating set GS is defined as exactly the set of flippable chords (those with $j \geq b'$). Thus, when the algorithm traverses the genealogical tree, it iterates directly over GS .

In our work, we define GS as *all* chords incident to the root, which is a superset of the flippable chords. When we need to generate children, we must still restrict to those with $j \geq b'$. However, we also need to check an additional condition: whether flipping the chord would create a multi-edge with chords from previously processed faces (in the biconnected setting). Therefore we maintain a separate **valid generating set** (VGS) that contains exactly those chords (v_1, v_j) satisfying:

1. $j \geq b'$ (flippability condition), and
2. The opposite pair of (v_1, v_j) is not already present in the global chord set S (safety condition).

This distinction is crucial for the pruning strategy described in Chapter 6 and for maintaining $O(1)$ amortised time per triangulation.

4.6 Summary

We have clarified the difference between the original definition of generating chords and our broader definition. The broader definition allows us to later filter chords based on multi-edge conflicts, leading to the concept of a valid generating set. The flippability condition $j \geq b'$ remains essential for preserving the parent-child relationship in the genealogical tree. This condition is used in our algorithm when iterating over candidate chords to flip.

Chapter 5

What the Paper Did and Why It Cannot Be Used for Biconnected Graphs

In this chapter we describe the algorithm of Parvez, Rahman, and Nakano [?] for generating all triangulations of a triconnected plane graph. We then explain why this algorithm fails when applied directly to biconnected graphs, focusing on the multi-edge problem caused by separation pairs.

5.1 The Parvez–Rahman–Nakano Algorithm for Triconnected Graphs

The algorithm for triconnected plane graphs follows the same high-level structure as the cycle algorithm presented in Chapter 3, but now multiple faces are processed simultaneously. Given a triconnected plane graph G with n vertices and k faces $F_1, F_2, \dots, F_k$, the algorithm constructs a *global genealogical tree* $\mathcal{T}(G)$ of triangulations of G .

5.1.1 Building the Root Triangulation

For each face F_i (a cycle), we choose a root vertex arbitrarily and construct the fan triangulation for that face. The root triangulation T_R of G is obtained by taking the union of these face triangulations. Since G is triconnected, the faces are independent: adding chords in one face does not affect the other faces.

The generating set GS^0 of the root triangulation is the union of the generating sets of the individual faces. Since each face with n_i vertices contributes $n_i - 3$ generating chords, the total number of generating chords in the root is $\sum_i (n_i - 3) = O(n)$ (by Euler's formula).

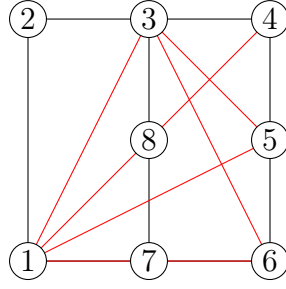


Figure 5.1: A root triangulation of a triconnected plane graph. The root vertex for each face is chosen independently; all chords are added simultaneously.

5.1.2 Generating Child Triangulations

From a triangulation T of G , a child triangulation T' is generated by flipping a generating chord in one eligible face. A face F_i is called **eligible** if all faces with smaller indices have already been processed; the algorithm processes faces in a fixed order.

The key property that enables constant-time generation is that flipping a generating chord in a face affects only that face. Because the graph is triconnected, the flipped chord cannot create a multi-edge with chords from other faces. Therefore, the algorithm can simply flip any generating chord and obtain a new valid triangulation.

5.1.3 Time and Space Complexity

The depth of the global genealogical tree is bounded by the total number of generating chords, which is $O(n)$. Each flip operation takes $O(1)$ time (updating the chord lists and opposite pairs for the affected face). Hence the algorithm generates all triangulations of G in $O(1)$ amortised time per triangulation, using $O(n)$ space.

5.2 Why the Algorithm Cannot Be Used for Biconnected Graphs

The authors of the original paper explicitly limited their algorithm to triconnected plane graphs. The reason is that in biconnected graphs, separation pairs can exist, and these cause a fundamental problem: the **multi-edge problem**.

5.2.1 Separation Pairs in Biconnected Graphs

Recall from Chapter 2 that a separation pair is a pair of vertices whose removal disconnects the graph. Biconnected graphs may contain separation pairs, while triconnected graphs do not.

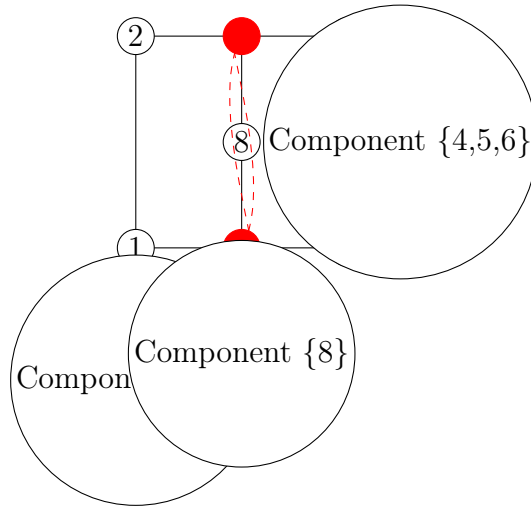


Figure 5.2: A biconnected plane graph with separation pair $(3, 7)$. Removing vertices 3 and 7 leaves three components: $\{1, 2\}$, $\{4, 5, 6\}$, and $\{8\}$.

5.2.2 The Multi-Edge Problem

In a triconnected graph, the triangulation of different faces are independent: any chord added in one face cannot appear in another face because that would require the two faces to share a common chord, which would create a separation pair. In a biconnected graph with a separation pair (a, b) , however, there exist two faces (one on each side of the separation pair) that share the same boundary vertices a and b .

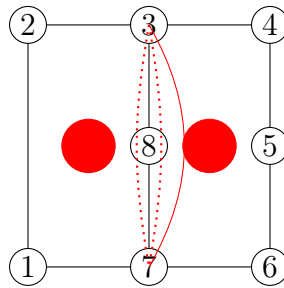


Figure 5.3: The separation pair $(3, 7)$ appears on two faces F_1 and F_2 . Triangulating each face independently may add the same chord $(3, 7)$ twice, creating a multi-edge.

If we independently triangulate the two faces using the algorithm of Parvez et al., it is possible that both faces receive the chord (a, b) (the chord that directly connects the separation pair). In a simple graph, having two copies of the same edge is not allowed. Thus the resulting structure would contain a multi-edge, which is not a valid triangulation of a simple graph.

5.2.3 Why the Original Algorithm Fails

The original algorithm was designed assuming that no multi-edge can appear. It processes all faces in parallel, flipping chords without checking whether the new chord already exists in another face. In a triconnected graph this assumption holds, but in a biconnected graph it does not. Therefore a direct application of the algorithm to a biconnected graph would generate many invalid triangulations containing multi-edges.

Moreover, simply discarding invalid triangulations is not sufficient because:

1. Checking for multi-edges would require additional time per flip, breaking the $O(1)$ amortised guarantee.
2. The number of invalid triangulations could be large, wasting substantial computational resources.
3. More importantly, the genealogical tree structure would be corrupted: a child might have a valid parent, but its descendants could all be invalid due to a persistent multi-edge. Pruning such branches requires careful analysis.

5.3 Conclusion

The Parvez-Rahman-Nakano algorithm is optimal for triconnected plane graphs but cannot be used directly for biconnected graphs because of the multi-edge problem caused by separation pairs. In the following chapters we develop a new approach that processes faces sequentially, uses a global chord set to detect conflicts, and prunes invalid branches safely, thereby extending the optimal complexity to biconnected graphs.

Chapter 6

Approaching Faces Differently – Pruning Strategy

In this chapter we develop a new approach to handle biconnected plane graphs by processing faces one after another and pruning branches that inevitably lead to multi-edges. We first explain why the original method of processing all faces simultaneously fails, then introduce a safe pruning strategy based on the observation that certain chords persist throughout an entire subtree of the genealogical tree.

6.1 Why We Cannot Process All Faces at Once

As argued in Chapter 5, the multi-edge problem arises when two faces share a separation pair and are triangulated independently. If we could guarantee that the same chord is never added in two different faces, the original algorithm would work. However, because we cannot a priori know which combinations of face triangulations are compatible, we must explore combinations systematically.

One naive approach would be to generate all possible triangulations for each face independently (using the cycle algorithm) and then combine them, discarding those that produce multi-edges. This would lead to an exponential blow-up because the total number of combinations is the product of the numbers of triangulations per face. Even worse, most combinations might be invalid, wasting time.

Therefore we need a method that explores the space of face triangulations in a coordinated way, building partial triangulations incrementally and backtracking when a conflict is detected.

6.2 Hashing Approach and Its Problems

A first idea is to maintain a global set S of all chords that have already been added during the construction of a partial triangulation. We could implement S as a hash set, which supports insertion, deletion, and membership queries in expected $O(1)$ time.

When we triangulate a new face, we could check each potential new chord against S before adding it. If a chord already exists, we could skip it or backtrack. However, this approach faces two major obstacles:

1. **Invalid triangulations would still be generated.** Even if we check the new chord, the process of flipping chords inside a face may create intermediate triangulations that are invalid (because they contain a chord already in S). Generating them wastes time, and discarding them after generation does not improve the asymptotic complexity.
2. **Checking validity takes more than $O(1)$ time per flip.** If we allow invalid triangulations to be generated, we must check not only the new chord but also whether any previously existing invalid chord has been removed. In general, this would require multiple hash lookups per flip, breaking the constant-time guarantee.

Thus a simple hash set without pruning is insufficient for achieving $O(1)$ amortised time per valid triangulation.

6.3 The Invalid Triangulation Problem

Suppose we generate a triangulation T of the current face that contains a chord e which already belongs to S (i.e., a chord from a previously triangulated face). Since we are only allowed to add chords that do not already exist, T is invalid. We could simply skip outputting T and continue. However, the genealogical tree of the current face would still include T and its descendants. If we do not prune the branch, we would generate many descendants, all of which might also be invalid because e persists.

The key question is: can we safely prune a branch at an invalid node without losing any valid triangulation that might appear deeper in that branch? If the answer is yes, we can avoid wasting time on entire subtrees.

6.4 Key Observation from the Cycle Algorithm (Persistent Chords)

Recall from Chapter 3 that in the genealogical tree of a cycle with root vertex v_1 , we only flip chords that are incident to v_1 (the generating chords). Chords that do not have v_1 as

an endpoint are never flipped; once they appear, they remain in all descendants of that node.

Lemma 6.1 (Persistence of non-root chords). *Let T be a triangulation of a cycle with root vertex v_1 , and let $e = (v_i, v_j)$ be a chord of T such that $i \neq 1$ and $j \neq 1$. Then for every descendant T' of T in the genealogical tree (i.e., every triangulation reachable from T by a sequence of flips of generating chords), the chord e belongs to T' .*

Proof. The only operation that removes a chord is flipping it. In the genealogical tree, we only flip generating chords, i.e., chords of the form (v_1, v_k) . Since e does not have v_1 as an endpoint, it is never flipped. Therefore, once e appears in a triangulation, it persists in all descendants. $\square$

This property is crucial for pruning: if a non-root chord e conflicts with the global set S (i.e., it creates a multi-edge), then every descendant triangulation of the current node also contains e and is therefore invalid. Thus we can safely prune the entire subtree rooted at T .

6.5 Safe Pruning Condition

Based on Lemma 6.1, we formulate a safe pruning condition.

Definition 6.1 (Prunable node). Let T be a triangulation of the current face F_i during a depth-first exploration of the global recursion tree. Let S be the global chord set containing all chords from previously processed faces $F_1, \dots, F_{i-1}$ and the partial triangulation of face F_i up to the current node. Suppose that T contains a chord e that does not have the root vertex v_r as an endpoint, and that e already belongs to S . Then T is called **prunable**, and the entire subtree rooted at T in the local genealogical tree can be skipped without missing any valid complete triangulation.

Justification. By Lemma 6.1, every descendant T' of T also contains e . Since $e \in S$ and e is already present in the global chord set (from a previous face), adding T' would create a multi-edge. Hence all descendants are invalid. No valid complete triangulation can be reached through this branch, so we may prune it. $\square$

6.6 Requirements for Safe Pruning

To implement this pruning strategy in a full algorithm for biconnected plane graphs, we must solve the following problems:

1. Process faces one by one in a fixed order so that the global set S is well-defined.

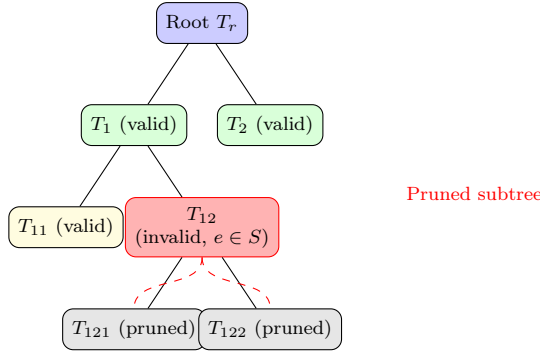


Figure 6.1: Safe pruning: node T_{12} contains a chord e already in S ; its entire subtree is pruned because all descendants would also contain e .

2. Ensure that chords from previous faces are never removed (they are permanent).
3. During the triangulation of a face, maintain a local genealogical tree where flips only affect chords incident to the root vertex.
4. Check, before flipping a generating chord, whether the new chord (the opposite pair) already exists in S . If it does, skip that flip (prune the branch).
5. When a chord that does not have the root vertex as an endpoint is added (as a result of a flip), we must check whether it conflicts with S . If it does, we prune the entire subtree from that node onward.
6. Guarantee that no valid triangulation is missed by this pruning.
7. Ensure that all operations (insertion, deletion, membership) on S take $O(1)$ expected time, and that the number of pruned nodes does not dominate the total work.

6.7 Outline of the Solution

In the following chapters we address these requirements:

- **Chapter 7** presents the multi-layer DFS that processes faces sequentially and maintains the global chord set S with push/pop operations for backtracking.
- **Chapter 8** discusses the problem of generating chords themselves causing multi-edges and introduces the concept of a *safe root vertex*.
- **Chapter ??** proves that a safe root always exists and gives an $O(n)$ algorithm to find one.
- **Chapter ??** establishes that each face yields at least $\Omega(n)$ valid triangulations, ensuring amortised constant time.

- **Chapter ??** introduces the valid generating set (VGS) and constant-time updates.

6.8 Summary

We have identified the persistence property of non-root chords as the key to safe pruning. If a triangulation of a face contains a chord that conflicts with previous faces, that chord will remain in all descendants, making the entire subtree invalid. Therefore we can prune such branches without losing any valid triangulation. This observation allows us to process faces sequentially while maintaining optimal complexity, provided we can also handle conflicts that involve generating chords themselves (which are root-incident and thus can be flipped away). The next chapter describes the multi-layer DFS that implements this pruning strategy.

Chapter 7

Multi-Layer DFS and Processing Faces One by One

In this chapter we describe the recursive tree structure that processes faces sequentially, building a global chord set incrementally and backtracking when no compatible triangulation exists for a face. We also formalise the pruning strategy introduced in Chapter 6 in terms of the opposite pairs of generating chords.

7.1 Processing Faces Sequentially

Let G be a biconnected plane graph with k faces $F_1, F_2, \dots, F_k$ (the order is arbitrary but fixed). We process the faces one after another, maintaining a global chord set S that contains all chords that have already been added by triangulations of earlier faces.

The overall structure is a **double-layer tree**:

- The outer layer is a recursion over faces: for each face F_i , we explore all its valid triangulations that are compatible with the chords in S (which come from faces $F_1, \dots, F_{i-1}$).
- The inner layer is the genealogical tree of triangulations of the current face F_i , as described in Chapter 3, but with additional pruning based on conflicts with S .

We denote a triangulation of face F_i by $t_{i,j}$, where j indexes the triangulations of that face in the order they are generated by the DFS of its local genealogical tree.

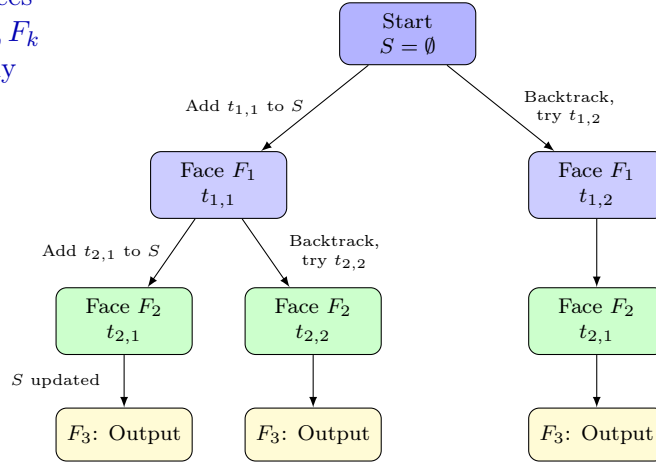
7.1.1 The Double-Layer Tree Structure

The exploration proceeds as follows:

1. Start with an empty global chord set $S = \emptyset$ and an empty partial triangulation $T = \emptyset$.

2. For face F_1 , build its genealogical tree (using the cycle algorithm). For each triangulation $t_{1,j}$ of F_1 :
 - Add the chords of $t_{1,j}$ to S (temporarily).
 - Move to face F_2 with the updated S .
3. For face F_2 , build its genealogical tree, but only consider those triangulations that do not create multi-edges with the chords already in S (i.e., no chord of the triangulation is already in S unless it is a boundary edge of the face). For each valid triangulation $t_{2,j}$:
 - Add its chords to S .
 - Move to face F_3 .
4. Continue recursively until the last face F_k . When a complete set of triangulations $t_{1,j_1}, t_{2,j_2}, \dots, t_{k,j_k}$ has been accumulated without conflicts, the union of all chords forms a valid global triangulation of G . Output this triangulation.
5. After exhausting all triangulations of the current face, backtrack: remove the chords of that face from S and try the next triangulation of the previous face.

Process faces
 $F_1, F_2, \dots, F_k$
 sequentially



For each face,
 explore all
 local tree
 triangulations

Backtrack:
 Remove chords
 from S

Figure 7.1: Double-layer DFS: outer recursion over faces, inner recursion over triangulations of each face. The global chord set S is updated when entering a branch (push) and restored when backtracking (pop). Each path from the root to a leaf corresponds to a complete valid triangulation of the graph.

7.1.2 Global Chord Set Management

The global chord set S is implemented as a hash set for expected $O(1)$ insertion, deletion, and membership queries. During the DFS:

- When we enter a node (i.e., when we commit to a particular triangulation $t_{i,j}$ of face F_i), we insert all chords of $t_{i,j}$ into S (except the boundary edges of the face, which are already present as edges of G).
- When we backtrack from that node (after exploring all its descendants), we remove those chords from S .

This ensures that S always contains exactly the chords from the current partial triangulation along the recursion path.

7.2 Pruning by Opposite Pair of Generating Chords

We now revisit the pruning condition introduced in Chapter 6 and express it in terms of the opposite pairs of generating chords.

Recall that in the local genealogical tree of a face with root vertex v_r , a child triangulation is obtained by flipping a generating chord (v_r, v_j) . The new chord added is the **opposite pair** $(v_o, v_{o'})$ of (v_r, v_j) . This new chord does **not** have v_r as an endpoint.

Lemma 7.1 (Pruning by opposite pair). *Let T be a triangulation of the current face F_i during the DFS, and let (v_r, v_j) be a generating chord of T whose opposite pair is $(v_o, v_{o'})$. If $(v_o, v_{o'})$ already belongs to the global chord set S (i.e., it was created by a previous face F_p with $p < i$), then the child triangulation $T' = \text{Flip}(T, (v_r, v_j))$ is invalid, and furthermore the entire subtree of the genealogical tree rooted at T' contains only invalid triangulations.*

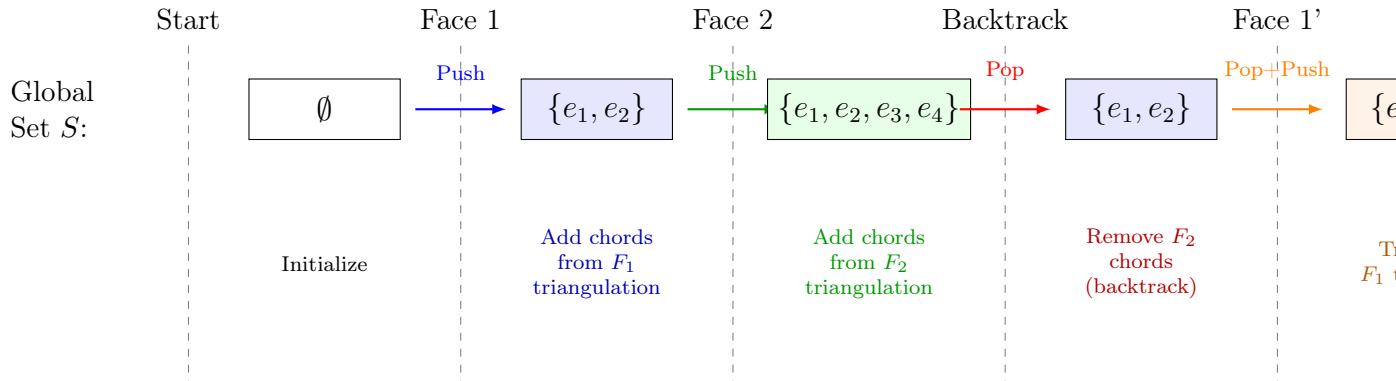
Proof. After flipping, the chord $(v_o, v_{o'})$ is present in T' . Because it does not have the root vertex v_r as an endpoint, Lemma 6.1 guarantees that $(v_o, v_{o'})$ persists in all descendants of T' in the local genealogical tree. Since $(v_o, v_{o'}) \in S$ and chords from previous faces are permanent, every descendant would contain a multi-edge. Hence the entire branch is invalid and can be safely pruned. $\square$

Therefore, when generating children of a triangulation T , we only flip those generating chords (v_r, v_j) for which the opposite pair is not already in S . All other generating chords are skipped (their branches are pruned at the root).

7.3 Backtracking When No Compatible Triangulation Exists

During the exploration of the genealogical tree of a face F_i , it is possible that no triangulation (including the root) is compatible with the current global set S . In that case, we backtrack to the previous face F_{i-1} and try its next triangulation.

More formally, the recursive procedure for a face F_i does the following:



Dynamic management: Push chords when exploring, Pop when backtracking
 S always reflects current path in recursion tree

Figure 7.2: Dynamic global chord set management. The set S is updated with push/pop operations during the DFS. Only chords that are not already in S can be added; otherwise the branch is pruned.

1. Find a safe root triangulation T_r for F_i (using the algorithm of Chapter ??).
2. If T_r conflicts with S (i.e., contains a chord already in S that is not a boundary edge), then no triangulation of F_i is compatible; return failure.
3. Otherwise, explore the local genealogical tree rooted at T_r , but only generate children that satisfy the pruning condition (opposite pair not in S).
4. For each valid child, update S (add its chords), recurse to F_{i+1} , and then backtrack (remove the child's chords).
5. After exhausting all valid children, return to the caller.

This backtracking mechanism ensures that we only explore combinations that have a chance of leading to a valid complete triangulation.

7.4 Correctness of the Multi-Layer DFS

The double-layer DFS is correct because:

- The outer recursion enumerates all possible combinations of face triangulations (by the completeness of the cycle algorithm for each face).
- The pruning condition (opposite pair already in S) only eliminates branches that are guaranteed to produce multi-edges (by Lemma 7.1). Hence no valid triangulation is missed.

- The use of a global chord set S with push/pop maintains the invariant that S always contains exactly the chords from the current partial triangulation along the recursion path.

7.5 Summary

We have described a multi-layer DFS that processes faces one by one, maintaining a global chord set S to detect conflicts. The pruning strategy based on opposite pairs of generating chords allows us to safely skip entire subtrees that would inevitably produce multi-edges. Backtracking ensures that we explore all compatible combinations without duplicates. In the next chapter we address the remaining problem: generating chords themselves may also create conflicts, requiring the concept of a *safe root vertex*.

Chapter 8

The Problem with Generating Chords and Multi-Edge Conflicts

In this chapter we address a critical issue that arises when some generating chords themselves create multi-edge conflicts with previously triangulated faces. We show that pruning such branches directly is unsafe because a generating chord can be flipped away in a child, potentially leading to valid descendants. Instead, we introduce the concept of a *safe root triangulation* and prove that such a triangulation always exists for any face, regardless of the chords already present from earlier faces.

8.1 Recap: Pruning Based on Non-Root Chords

In Chapter 6 we established that if a triangulation of a face contains a chord e that does not have the root vertex as an endpoint, and e already belongs to the global set S (from previous faces), then the entire subtree rooted at that triangulation can be safely pruned. This is because e persists in all descendants (Lemma 6.1) and will always cause a multi-edge.

Thus, when exploring the local genealogical tree of a face, we only need to check, after each flip, whether the newly added chord (which never has the root as an endpoint) conflicts with S . If it does, we prune the branch immediately.

8.2 The Problem with Generating Chords

Now consider a generating chord (v_r, v_j) — a chord that has the root vertex v_r as an endpoint. If (v_r, v_j) itself is already present in S , then adding it would create a multi-edge. However, unlike non-root chords, a generating chord can be flipped away in a child triangulation, so it does not persist in all descendants. Therefore, we cannot prune the branch simply because the generating chord conflicts with S ; the conflict might be

resolved by flipping that chord later.

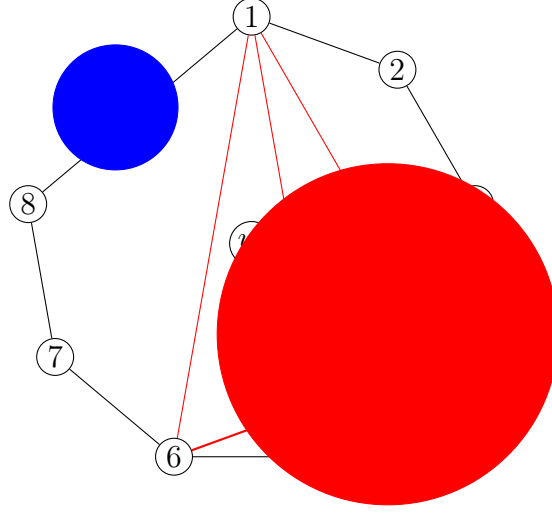


Figure 8.1: A generating chord (v_r, v_j) might already exist in S from a previous face. Unlike a non-root chord, it can be flipped away in a child, so the branch may still contain valid triangulations. Thus we cannot prune based on a conflicting generating chord alone.

Consequently, to guarantee that every branch we explore can lead to valid triangulations, we must ensure that the *root triangulation* of the face (the starting point of the local genealogical tree) does not contain any chord that conflicts with S . In other words, we need a **safe root triangulation**.

8.3 The Concept of a Safe Root Triangulation

Definition 8.1 (Safe root vertex). Let F be a face (cycle) with vertices listed in counterclockwise order, and let S be the global chord set from previously processed faces. A vertex r of F is called a **safe root vertex** if, when we construct the fan triangulation from r (i.e., add all chords (r, v) for every vertex v of F that is not a neighbour of r on the cycle), none of these chords belongs to S . The corresponding fan triangulation is called a **safe root triangulation**.

If we can find a safe root vertex for a face, then the root of its local genealogical tree is guaranteed to be compatible with all previously added chords. Moreover, because the root has the maximum number of generating chords, any conflict that appears later (in a non-root chord) can be safely pruned. Thus the existence of a safe root is essential for our algorithm.

8.4 Two Fundamental Questions

The above discussion raises two immediate questions:

1. **Existence:** For any face in a biconnected plane graph, given an arbitrary set S of chords from previous faces (which are all inside the outer region of the face), does there always exist a safe root vertex? If not, our algorithm would fail for some inputs.
2. **Efficiency:** Assuming a safe root exists, can we find one in time $O(n)$ (where n is the number of vertices of the face) without harming the overall $O(1)$ amortised time per triangulation?

The remainder of this chapter is devoted to answering the first question positively: we prove that every face in a biconnected plane graph contains at least two safe root vertices, regardless of the chords already present. The second question (efficiently finding a safe root) is addressed in Chapter ??.

8.5 Existence of a Safe Root Vertex

We now prove that, under the planarity assumption, there is always at least one (in fact, at least two) vertex that can serve as a safe root.

Theorem 8.1 (Existence of safe root vertices). *Let F be a face (a simple cycle) of a biconnected plane graph G , and let S be any set of chords that lie strictly outside F (i.e., in the region of the graph that has already been triangulated by previous faces). Then there exist at least two vertices of F that are safe roots.*

Proof. We prove the theorem by induction on the number of vertices of F and on the structure of chords outside F . The key observation is that any chord in S that connects two vertices of F must lie entirely outside F (by planarity) and divides the cycle into two arcs. Such a chord imposes constraints on which vertices can be safe roots.

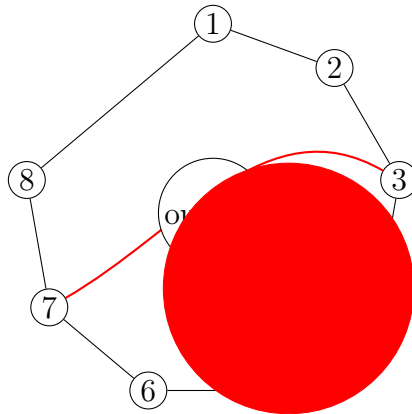


Figure 8.2: A chord (v_i, v_j) outside the face divides the cycle into two arcs. No chord can cross this chord by planarity, so constraints are separated.

Let the vertices of F be $v_1, v_2, \dots, v_m$ in counterclockwise order. If there is no chord in S connecting two vertices of F , then every vertex is a safe root, and the theorem holds trivially.

Otherwise, consider a chord $(v_i, v_j) \in S$ with $i < j$. This chord lies completely outside F (by definition of S as chords from previous faces, which are on the other side of F). By planarity, any other chord in S that connects two vertices of F must lie entirely on one of the two arcs defined by (v_i, v_j) ; it cannot cross (v_i, v_j) . Thus (v_i, v_j) separates the cycle into two independent subproblems.

The two arcs are:

- The arc $A_1 = v_i, v_{i+1}, \dots, v_j$ (including both endpoints)
- The arc $A_2 = v_j, v_{j+1}, \dots, v_i$ (wrapping around)

Each arc, together with the chord (v_i, v_j) , forms a closed region (a smaller cycle) that is independent of the other. Moreover, any chord in S that lies inside A_1 (i.e., with both endpoints in A_1) can be considered as an external chord for the subface bounded by A_1 and the chord (v_i, v_j) . The same holds for A_2 .

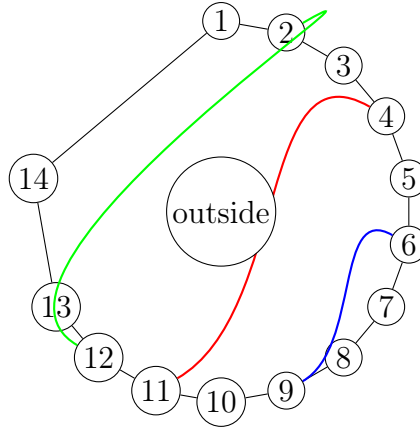


Figure 8.3: Multiple external chords partition the face into independent segments. Each segment can be treated separately; a safe root can be found in each segment, and the endpoints of the segment are potential safe roots for the whole face.

By recursively applying this decomposition, we reduce the problem to finding a safe root in a smaller face that has no external chords (a “primitive” segment). Therefore it suffices to prove that every primitive segment (a cycle with no chords from S inside it) contains at least two safe roots.

Consider a primitive segment consisting of t vertices $u_1, u_2, \dots, u_t$ in cyclic order, with the additional chord (u_1, u_t) being the boundary of the segment (this chord may be an original edge of G or a chord from S). Inside this segment there are no chords from S . We claim that at least two vertices among $\{u_2, \dots, u_{t-1}\}$ are safe roots.

The proof proceeds by induction on t .

Base case: $t = 3$ (a triangle). The segment has only three vertices. The chord (u_1, u_t) is already present. The interior vertex u_2 has degree 2 within the segment (connected to u_1 and u_3), so it cannot have any chord to a non-neighbour (there are none). Thus u_2 is a safe root. Similarly, if the triangle is isosceles, both u_1 and u_3 are also safe? Actually u_1 and u_3 have the chord (u_1, u_3) already present, so they are not safe because adding (u_1, u_3) would create a multi-edge. So only u_2 is safe, but we need two? Wait, the theorem claims at least two safe roots for the whole face, not necessarily for a primitive segment. For a primitive segment that is a triangle, there is exactly one vertex that is not an endpoint of the external chord. That vertex is safe. However, the whole face consists of many such segments concatenated; the endpoints of segments (which are vertices of the original face) may be safe when considering the whole face. The induction will accumulate safe roots.

Actually, the standard proof (as in the author's original text) uses the notion of T_n structures. Let us follow that argument directly.

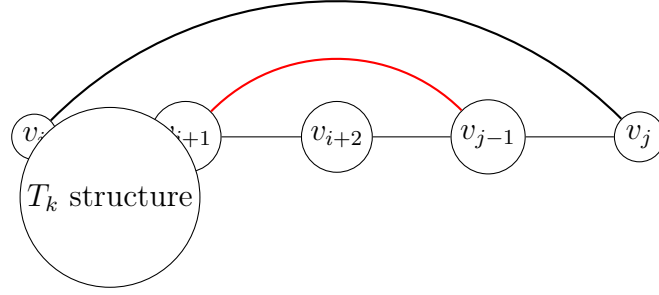


Figure 8.4: A T_k structure: a path of vertices with a chord connecting the endpoints, and possibly additional chords inside. The induction shows that at least one vertex among the interior ones is safe.

Define a T_k structure as a set of vertices $\{v_i, v_{i+1}, \dots, v_j\}$ such that:

- The boundary edges $(v_i, v_{i+1}), \dots, (v_{j-1}, v_j)$ are part of the cycle or already added chords.
- There is a chord (v_i, v_j) in S (or a boundary edge of the segment) that lies outside the face.
- All chords from S with both endpoints in this set lie strictly inside the region bounded by the path and the chord (v_i, v_j) .

A T_k structure has $k + 1$ vertices? In the original text, T_n refers to a structure with n vertices along the path. We prove by induction on the number of interior vertices that there is at least one safe root.

Base case: T_1 structure. This consists of three vertices v_i, v_{i+1}, v_j with $j = i + 2$. The vertex v_{i+1} has no chords to any non-neighbour (its only neighbours are v_i and v_j), so it is a safe root.

Inductive step: T_k structure with $k \geq 2$. Consider the vertex v_{i+1} . Its neighbours on the path are v_i and v_{i+2} . It could have chords to vertices $v_{i+3}, v_{i+4}, \dots, v_j$ (since these are non-neighbours). There are two cases:

1. v_{i+1} **has no chord to any vertex beyond v_{i+2} .** Then v_{i+1} itself is a safe root, because none of the chords from v_{i+1} to non-neighbours are present in S . The chord (v_i, v_j) does not involve v_{i+1} , so no conflict arises.
2. v_{i+1} **has a chord to some vertex v_p with $p \geq i + 3$.** This chord, together with the existing chord (v_i, v_j) , creates a smaller $T_{k'}$ structure with $k' < k$ (for example, between v_{i+1} and v_p or between v_p and v_j). By the induction hypothesis, this smaller structure contains a safe root. Moreover, that safe root is also a safe root for the original T_k structure, because any chord from it to a non-neighbour would have to lie inside the smaller structure and thus is already accounted for.

In both cases, a safe root exists within the T_k structure.

Since the whole face can be partitioned by the chords in S into a sequence of $T_{k_1}, T_{k_2}, \dots, T_{k_m}$ structures (each corresponding to a maximal interval of vertices between two consecutive external chords), and each T_{k_ℓ} structure contains at least one safe root, the entire face contains at least m safe roots. Because the face is a cycle, there are at least two such intervals (one on each side of any external chord), so $m \geq 2$. Hence there are at least two safe root vertices for the whole face. $\square$

Remark 8.1. The proof is constructive: it shows how to locate safe roots by examining the structure of external chords. In Chapter ?? we give an explicit $O(n)$ algorithm to find one safe root without explicitly constructing the T_k decomposition.

8.6 Implications for the Algorithm

Theorem 8.1 guarantees that we can always start the triangulation of a face with a safe root triangulation, provided we have correctly maintained the global chord set S of previously added chords. Thus the root of the local genealogical tree will be compatible with all earlier faces. Any conflict that later appears will involve a non-root chord (since generating chords are, by construction, all safe at the root), and such conflicts can be safely pruned as described in Chapter 6.

The remaining task is to find a safe root vertex efficiently (in $O(n)$ time) so that the total building cost per face does not exceed the number of triangulations generated. This is the subject of Chapter ??.

8.7 Summary

We have identified a critical issue: generating chords themselves can conflict with previously added chords, but we cannot prune branches based on such conflicts because generating chords can be flipped away. To overcome this, we introduced the concept of a *safe root triangulation* — a root of the local genealogical tree whose generating chords are all conflict-free. We proved that every face in a biconnected plane graph contains at least two safe root vertices, regardless of the chords already present from previous faces. This existence result ensures that our algorithm can always initialise the local tree safely. The next chapter provides an efficient method to find a safe root in linear time.